PROPOSAL — Interactive schematics: editor + real-time circuit simulator + logic + net-extraction

How EveryCircuit / Scheme-it / LucidChart / Falstad CircuitJS actually work, and how to bring it into THE VIEWER — grounded, offline, on the current specs.

1 · HOW THE REFERENCE TOOLS WORK

- **EveryCircuit** — Real-time custom MNA engine + nonlinear device models. Transient/AC/DC. Tune a knob and it re-solves live; current animates as moving charges; per-node waveform scopes; digital wires colour-coded.
- **Falstad CircuitJS1** — Open-source (GPLv2) browser simulator (GWT->JS) with OFFLINE standalone builds. Same live animation + scopes + analog & digital. Embeddable via iframe — the fastest path to a working sim.
- **Digi-Key Scheme-it** — Browser schematic CAPTURE: 700+ symbols (+millions via catalog), BOM manager, export SVG/PDF/KiCAD. Drawing + parts, NOT a simulator.
- **LucidChart / Scheme-it style editor** — Drag-place symbols, wire them, snap-to-grid, properties — the 'digitally manipulatable schematic' authoring surface.
- **SINA / Netlistify (research)** — Deep-learning image->netlist: component detection + connectivity (CCL) + OCR + VLM. SINA ~96% accuracy, open-source. This is the BRIDGE from a raster TM schematic to a simulatable netlist.

2 · ARCHITECTURE FOR THE VIEWER (three layers + a bridge)

A · SCHEMATIC EDITOR / CAPTURE

- Canvas (SVG/Canvas) + symbol library
- drag-place parts · draw/route wires · grid snap
- properties panel + values + BOM
- data model = a NETLIST (nodes, components)
- export SVG / PDF / netlist
- FULLY OFFLINE, doable now

B · REAL-TIME SIMULATOR

- engine solves the netlist each step
- tune a parameter -> instant re-solve
- animate current (moving dots) + node colours
- scopes: per-node V/I waveforms
- DC · AC sweep · transient
- Option 1: embed CircuitJS (GPLv2, fast)
- Option 2: custom MNA engine (no-GPL, tight)

C · DYNAMIC LOGIC RENDERING

- digital gates / flip-flops / buses
- event-driven 0/1 propagation (light)
- colour-coded logic states + bus values
- clock stepping + timing-diagram scopes
- truth tables on hover
- pairs with the analog engine

All three are client-side JS — no server load; the RTX 4050 is far more than enough (the sim is light).

3 · HOW THE REAL-TIME SIM WORKS (the numerical heart — EveryCircuit / SPICE / CircuitJS)

1 Build the matrix — Modified Nodal Analysis: assemble G·v = i from the netlist (one equation per node). ▶

2 Stamp components — Each R/L/C/diode/transistor/source 'stamps' its contribution; L/C use companion models per timestep. ▶

3 Solve (nonlinear) — Newton-Raphson iterates until the nonlinear devices (diodes, transistors) converge. ▶

4 Integrate time — Trapezoidal / backward-Euler advance one dt; repeat ~real-time for a smooth animation. ▶

5 Animate + tune — Map node voltages->colours, branch currents->moving-dot speed; a slider edits a value and re-solves live.

4 · THE BRIDGE — from a real TM schematic to a simulatable netlist

raster TM schematic — the real scanned/vector page in the library ▶

detect components — vision model (YOLO-style) finds relays, resistors, connectors... ▶

trace connectivity — line-following / connected-components builds the wires + nodes ▶

OCR + VLM labels — ref-designators, values, pin IDs assigned (SINA-style) ▶

netlist — -> load straight into the editor + simulator

On the 4050: inference works for component/line detection. Bigger models + batching all 1,093 schematics + any fine-tuning -> the desktop/RTX 5070 (12 GB). This is the one 'needs-power' piece.

THE GROUNDING LINE

The editor + simulator are GROUNDED when YOU build/edit a circuit — your input is the source of truth (great for training: 'what happens if this relay coil opens?'). An AUTO-extracted netlist from a TM raster MUST be shown for review/edit and labelled 'auto-extracted — verify against the TM' before it's trusted; the cited TM page always sits alongside. We never present a simulated result as the manual's ground truth without that human check.

RECOMMENDED PATH

1) Build the schematic EDITOR (capture + netlist + BOM, offline) — immediate value. 2) Add the SIMULATOR by embedding Falstad CircuitJS (GPLv2, offline) to validate UX fast, then optionally a custom MNA engine for tight integration + no-GPL distribution. 3) Logic rendering. 4) The net-extraction BRIDGE last (the power feature). All on the 4050 except heavy extraction.